ClientPulse AI

Complete System Architecture, Module Workflow, and Technical Specification

Version: 1.0.0 Production | **Author:** DeepMind Advanced Agentic Engineering Team | **Date:** November 2026

1. Executive Summary & Purpose

ClientPulse AI is a mission-critical, enterprise client communication and follow-up management SaaS platform. In modern professional service environments (consulting, legal, engineering, agency, software development), client project deliverables frequently stall because teams lose track of pending requests, deliverables owed by clients, or urgent blocker inquiries. ClientPulse AI bridges this operational gap by continuously tracking waiting periods, analyzing conversation intent using Google Gemini AI, calculating deterministic SLA urgency, and enabling employees to review, approve, and dispatch follow-ups seamlessly across email and self-serve client portals.

2. High-Level System Architecture

ClientPulse AI utilizes a decoupled modern 3-tier architecture: a reactive Single-Page Application (SPA) frontend, a high-concurrency asynchronous FastAPI REST backend, and a deterministic relational persistence layer with automated fallback.

```
+-------------------------------------------------------------------------------------------------------------+ | CLIENTPULSE AI SYSTEM ARCHITECTURE |
+-------------------------------------------------------------------------------------------------------------+ | | [ FRONTEND TIER (Vite + React 18 + TypeScript) ] [ PUBLIC CLIENT PORTAL ] | | - Employee Dashboard with real-time SLA metrics - Standalone mobile-friendly UI | | - Client, Contact & Project Management Views - Zero-login token-based access | | - Two-Way Messaging Thread & AI Follow-Up Modal - Two-way project messaging | | - JWT Bearer Authentication Session Storage - Internal notes/budgets hidden | | | | HTTPS REST / JSON | | v | | [ BACKEND API LAYER (FastAPI + Python 3.12 + Uvicorn) ] | |
+--------------------+--------------------+--------------------+--------------------+ | | | /api/v1/auth | /api/v1/clients | /api/v1/projects | /api/v1/portal | | | | - JWT session sign | - CRUD validation | - Project linking | - Token validation | | | | - Password verify | - Deduplication | - Token rotation | - Safe responses | | | +--------------------+--------------------+--------------------+--------------------+ | | | /api/v1/followups | /api/v1/ai | /api/v1/emails | /api/v1/contacts | | | | - Deterministic SLA| - Gemini Assistant | - Provider factory | - Channels & phone | | | | - Snooze/Escalate | - Tone drafting | - Inbound webhooks | - Consent & opt-out| | |
+--------------------+--------------------+--------------------+--------------------+ | | | | [ BACKGROUND WORKER ENGINE ] [ AI TELEMETRY & NLP SERVICE ] | | - Periodic 60s async evaluation loop - Google Gemini 1.5 Flash REST API | | - Fractional day calculation math - Context synthesis & fallback logic | | | | | SQLAlchemy 2.0 ORM | | v | | [ PERSISTENCE & STORAGE TIER ] | | - PostgreSQL (Production Database) <---> SQLite (Autonomous Dev Fallback clientpulse_dev.db) | | - Tables: clients, projects, project_links, conversations, messages, followups, contacts, emails |
+-------------------------------------------------------------------------------------------------------------+
```

3. Detailed Module-by-Module Breakdown

Each module in ClientPulse AI was engineered for a specific operational necessity, with concrete data contracts, security boundaries, and user workflows.

Module 1: Employee Authentication & Authorization (/api/v1/auth)

Why it is used: Enterprise systems cannot expose proprietary project metadata, internal deliverables, client contact databases, or financial budgets to unauthenticated parties or client portal viewers.
How it works: Employees authenticate using their credentials via POST /api/v1/auth/login. The backend verifies the password hash, signs a cryptographic JWT access token with an expiration timestamp and employee role, and issues it to the frontend. The frontend stores this token in secure session storage and injects it as an HTTP Authorization: Bearer <token> header on all subsequent internal endpoints.
Key Files: backend/app/api/auth.py, backend/app/core/auth.py, frontend/src/components/auth/EmployeeLoginModal.tsx

Module 2: Client & Contact Management (/api/v1/clients & /api/v1/contacts)

Why it is used: Provides a single source of truth for company accounts and multi-stakeholder contacts. Prevents duplicate company accounts and manages channel preferences (Email, WhatsApp, Phone, Portal).
How it works: Validates uniqueness of corporate email addresses, enforces international telephone standards, tracks consent status (opted_in, pending, opted_out) for GDPR/CAN-SPAM compliance, and associates primary and secondary organizational contacts.
Key Files: backend/app/models/client.py, backend/app/models/contact.py, frontend/src/views/ClientsView.tsx, frontend/src/views/ContactsView.tsx

Module 3: Project Management & Tokenized Portal Links (/api/v1/projects)

Why it is used: Projects house deliverable milestones, budgets, and deadlines. Clients must be able to view and participate in their projects without creating friction-heavy employee login accounts.
How it works: Generates cryptographically unguessable URL-safe tokens (e.g. 43-character entropy) linked to each project. Generating a new link automatically revokes any previous link. Employees can explicitly revoke client links at any time to terminate external access immediately.
Key Files: backend/app/models/project.py, backend/app/models/project_link.py, frontend/src/components/common/ProjectShareLinkModal.tsx

Module 4: Public Client Communication Portal (/api/v1/portal)

Why it is used: Gives clients a responsive, clean, branded view of their active project and previous discussions on desktop or mobile with zero authentication overhead.
How it works: Validates the unguessable URL token. If active, serves the project title, description, and message thread.
Privacy Filter: Budgets, internal employee notes, and internal risk scores are strictly excluded from the portal schema. Clients can submit messages which are stored with sender_type='client' and instantly trigger follow-up SLA recalculations.
Key Files: backend/app/api/portal.py, backend/app/schemas/portal.py, frontend/src/views/ClientPortalView.tsx

Module 5: Deterministic Follow-Up & SLA Engine (app.services.followup_engine)

Why it is used: Human employees forget to check threads when waiting for client approvals. Without an automated SLA tracker, projects suffer delivery delays and client churn.
How it works: Calculates waiting time in fractional days between the last relevant message timestamp and now_utc. Enforces deterministic state transitions:

- **Pending (< 2.0 days):** On track. Priority: Low.
- **Due (>= 2.0 days):** Recommended follow-up needed. Priority: Medium.
- **Overdue (>= 5.0 days):** Client blocker. Priority: High.
- **Escalated (>= 7.0 days):** Executive escalation. Priority: Urgent.
- **Snoozed:** Temporarily suppressed until a specified date.
- **Completed:** Deliverables received and thread resolved.

Critical Rule: Sending a reminder restarts the waiting period to 0.0 days but does **not** mark the item completed until client reply is established.
Key Files: backend/app/services/followup_engine.py, backend/app/models/followup.py

Module 6: Google Gemini AI Conversation Analysis & Draft Generator (/api/v1/ai)

Why it is used: Enables employees to comprehend complex multi-turn threads in seconds and draft empathetic, professional, and highly context-aware reminder messages without manual copywriting.
How it works: Extracts the complete chronological message history and feeds it to the Gemini 1.5 Flash API with structured JSON output instructions. Extracts: (1) Pending action, (2) Expected responder, (3) Requested deliverables, (4) Expected response date, (5) True completion status, (6) Project risk impact.
Safety & Fallback: If no internet or API key is available, an intelligent local rule-based fallback executes automatically without crashing.
Key Files: backend/app/services/gemini_service.py, backend/app/api/ai.py, frontend/src/components/common/GenerateFollowUpModal.tsx

Module 7: Email Service Abstraction & Inbound Webhooks (/api/v1/emails)

Why it is used: Many clients communicate via native email clients rather than portals. The system must support outbound sending and inbound reply ingestion without coupling to a single email vendor.
How it works: Features a Provider Abstraction Factory supporting Mock (safe development), SMTP, Resend, and SendGrid. Inbound replies hit authenticated webhooks (POST /api/v1/emails/webhook/inbound) which match client email addresses and automatically append incoming responses to conversation threads.
Key Files: backend/app/services/email/factory.py, backend/app/api/emails.py, frontend/src/components/email/EmailOutboxTab.tsx

4. Complete End-to-End Operational Workflow

The following 14-step sequence illustrates the lifecycle of a client engagement in ClientPulse AI, from onboarding to deliverable confirmation:

Phase	Workflow Description
Step 1: Employee Authentication	Employee logs into ClientPulse AI with JWT credentials. All client data access is authenticated.
Step 2: Client Account Creation	Employee creates client record (Name, Company, Email, Phone). Duplicate emails are blocked.
Step 3: Project Specification	Employee sets up project with deliverables, deadline, and financial budget.
Step 4: Unique Portal Link Generated	System generates a high-entropy share URL. Internal notes and budgets are kept confidential.
Step 5: Client Interaction	Client opens portal on mobile/desktop without an account and posts a kickoff question.
Step 6: Employee Request Sent	Employee replies requesting required architecture diagrams and sign-off by Friday.
Step 7: AI Detects Pending Action	AI identifies that the client owes deliverables and estimates a 48-hour response window.
Step 8: Waiting Time Tracking	Waiting time elapses (e.g. 2.5 days). Follow-up status transitions from 'Pending' to 'Due'.
Step 9: Employee Alerted on Dashboard	Dashboard highlights the conversation as a follow-up required item.
Step 10: AI Follow-Up Generation	Employee clicks 'Generate Follow-Up'. AI synthesizes a professional draft tailored to the context.
Step 11: Employee Review & Approval	Employee reviews, edits, and approves draft. Reminder is logged into thread history.
Step 12: Waiting Window Restarts	Waiting time resets to 0.0 days. System remains in pending status awaiting client feedback.
Step 13: Client Delivers Assets	Client uploads or messages the requested approval and deliverables into the portal.
Step 14: Action Resolution	AI detects completion words ('approved', 'attached'). Employee marks follow-up officially completed.

5. Database Schema & Relational Model

```
[clients] id, name, email (unique), company, phone, status, is_archived, notes, timestamps |-- 1:N -->
[projects] id, client_id, title, description, status, budget, deadline, timestamps | |-- 1:N -->
[project_links] id, project_id, token (unique), is_active, revoked_at | |-- 1:N --> [conversations] id,
client_id, project_id, subject, channel, status | |-- 1:N --> [messages] id, conversation_id, sender_type,
content, sent_at | |-- 1:1 --> [followups] id, conversation_id, status, priority, days_waiting |-- 1:N -->
[contacts] id, client_id, project_id, name, email, phone, preferred_channel, consent |-- 1:N -->
[email_messages] id, client_id, contact_id, direction, status, idempotency_key, body
```

6. Production Startup & Operations Guide

Backend Server (Port 8000):
.\backend\.venv\Scripts\python.exe -m uvicorn app.main:app --app-dir backend --host 0.0.0.0 --port 8000 --reload

Frontend Development Server (Port 5173):
npm --prefix frontend run dev

Automated Test Suite:
.\backend\.venv\Scripts\python.exe backend/test_e2e_workflow.py

Demo Employee Sign-in: employee@clientpulse.ai / ClientPulse2026!